BHASHASETU

Comprehensive Product & Engineering Audit

Domain Review • Phased Roadmap • Future Features

Repository: github.com/reason706/BhashaSetu-
Branch reviewed: main
Prepared: 6 November 2026

Source-based review; runtime checks not executed

Executive summary

BhashaSetu is a feature-rich English-learning application designed around Nepali-speaking learners. The repository combines a React/TypeScript frontend, a substantial curriculum and vocabulary corpus, browser speech capabilities, local persistence, Firebase authentication/Firestore, PWA support, and Gemini-backed coaching endpoints. Its strongest differentiator is the learner-specific content and practice design—not simply the number of screens.

The central recommendation is to shift the next development cycle from feature accumulation to reliability, evidence-based learning outcomes, security controls, accessibility, and maintainability. Several high-impact implementation concerns are visible in source: the event-sync endpoint acknowledges events without durable storage; AI endpoints do not show a server-side authentication/rate-limit layer; and test infrastructure appears thin relative to the breadth of the app.

Audit confidence: medium for source-level observations; low for runtime, deployment, and real-user outcome claims. No production build, live browser session, Firebase Emulator run, or device test was executed in this environment. This report distinguishes source evidence from items requiring verification.

Audit scope, approach & limitations

Repository reviewed: https://github.com/reason706/BhashaSetu- (main branch). The repository tree and selected files were inspected through GitHub repository access. Review targets included README, package.json, App.tsx, server.ts, Firebase service, storage service, curriculum and lexicon files, selected practice components, PWA hook, Firestore rules and security documentation.

- Source review: architecture, routes, data models, state handling, storage, AI request patterns, and configuration.
- Feature review: syllabus, dictionary, SRS, roleplay, shadowing, dictation, minimal pairs, articulation, writing linter, printable sheets, daily challenges and analytics as documented/located in source.
- Risk review: public API exposure, data loss paths, authentication assumptions, request validation, error handling, offline state and test evidence
- Product review: learner journeys, discoverability, practice feedback, inclusion, measurement and future feature fit

Not performed: npm install/build/lint; live application walkthrough; browser accessibility scan; screen-reader test; API penetration test; Firebase Emulator execution; Lighthouse; load test; deployed environment review; content fact-check at corpus scale. Findings requiring those steps are marked as validation work.

Phase 1 — Product & repository baseline

Strengths — The product has a clear audience and coherent instructional premise: English learning for Nepali speakers, with attention to pronunciation contrasts, silent-letter patterns, vocabulary, grammar interference and workplace communication. README describes a sequential B1–C1, 12-lecture curriculum and multiple practice tools.

Risks / gaps — README and package metadata are not fully polished (package name/version remain generic). Documentation describes ambitious feature claims; a feature inventory is not equivalent to proof of correct behavior. Keep docs aligned with what is actually shipped and tested.

Recommendations — Create a product capability matrix with status labels (shipped, experimental, browser-dependent, API-required). Add a short architecture diagram, deployment guide, environment-variable reference, data lifecycle explanation and support/troubleshooting page.

Evidence — README.md; package.json; src/data/syllabus.ts; src/data/lectureEnhancements.ts; src/data/massiveLexicon.ts

Phase 2 — UI/UX, navigation & accessibility

Strengths — The editorial warm-paper/dark-ink design language, semantic silent-letter highlighting, IPA typography and Nepali script support create a distinctive identity. The application includes focused learning surfaces rather than a generic chatbot-only experience.

UX risks — A broad feature set can make the first session overwhelming. Multiple tools, modals and practice modes can compete for attention. The audit did not run a live usability session, so these are design risks to test—not confirmed usability failures

Recommendations — 1) Introduce a guided first-session path: choose goal/level → short diagnostic → first lesson → practice → progress recap. 2) Make a single primary action prominent on Home. 3) Group tools by learner goal (Hear, Speak, Read, Write, Review). 4) Add consistent empty/loading/error states. 5) Keep navigation labels predictable and avoid modal stacking

Accessibility checklist — Use semantic landmarks and heading order; visible focus; keyboard access for every control; no color-only silent-letter cues; sufficient contrast in both themes; reduced-motion support; accessible names for icon buttons; labels/errors for forms; announce asynchronous feedback; ensure Devanagari fonts render at comfortable sizes; test zoom/reflow at 200–400%

Validation — Run axe-core or Lighthouse accessibility checks, then manual keyboard and VoiceOver/NVDA tests. Test at 320px width, tablet, desktop, landscape, large text and dark mode. Conduct 5–8 learner usability sessions and record task completion/time/confusion.

Phase 3 — Architecture & maintainability

Strengths — The codebase has useful boundaries: components, data, hooks, services, types and utilities. Curriculum types are centralized. Firebase and local storage are not entirely embedded in visual components

Main concern — App.tsx appears to coordinate many domains—navigation, progress, authentication, sync, dashboard interactions, AI actions, search and modal state. This concentration creates a change-risk hotspot and makes feature tests harder

Target structure — Move toward feature-oriented modules: app shell/providers; features/dashboard; curriculum; dictionary; practice; progress; auth; AI coach; shared UI; domain services. Keep domain state and effects close to their owning feature

Refactoring sequence — Extract one low-risk feature at a time; preserve current behavior with characterization tests; introduce typed service interfaces; avoid a big-bang rewrite. Separate UI state from persisted learner state and use selectors for derived values

Code hygiene — Rename package metadata; remove stale backups only after confirming their purpose; establish naming conventions; document generated/curated data; add lint rules and formatting; review large component files for responsibility boundaries

Evidence — src/App.tsx; src/services/firebase.ts; src/services/storage.ts; src/types/curriculum.ts; backup/

Phase 4 — Feature-by-feature product audit

Curriculum / lesson player — A sequenced B1–C1, 12-lecture curriculum is a strong scaffold. Improve with explicit learning objectives, estimated duration, prerequisite links, lesson completion criteria, spaced revisit prompts, and content-version tracking. Test whether learners can resume exactly where they stopped

Dictionary / lexicon — Large vocabulary corpus is a differentiator. Search/filter performance, duplicate/variant handling, IPA consistency, definitions, translations and source quality need automated data validation. Clarify whether generated words are pedagogically valid or simply algorithmically derived; prevent low-quality derived entries from appearing as authoritative

Spaced repetition (SRS) — SRS exists in storage logic. Current scheduling logic appears rule/streak based (learning/review/mastered intervals) rather than a validated memory model. Consider Again/Hard/Good/Easy ratings, due queue, missed-day recovery, per-word history, and transparent 'why this is due' explanation. Test timezone/day-boundary, incorrect answers, repeated sessions and merge conflicts

Minimal pairs & articulation — The /v/–/w/ contrast and articulatory cues are especially relevant for Nepali speakers. Ensure explanations acknowledge accent variation and avoid implying one native-like accent is the only valid target. Add calibrated audio examples, phoneme-specific feedback and accessible visual alternatives

Dictation & shadowing — These tools can support listening and fluency, but browser TTS is not a consistent reference voice across devices. Label voice as device-provided; allow speed/voice selection; provide transcript and replay controls; never penalize learners based on microphone/environment noise

Writing interference linter — Regex-based rules are useful as explainable hints, but can generate false positives/false negatives and miss context. Label suggestions as 'possible revision'; show rule rationale; let learners accept/reject; include examples and exceptions; collect anonymized opt-in correction feedback

Roleplay / AI coach — Structured outputs and scenario roleplay are valuable. Add conversation reset/export, learner-controlled feedback intensity, response streaming if useful, prompt-injection boundaries, moderation/abuse handling, timeouts, token limits, retry logic and a visible AI disclaimer. Validate output schema server-side before returning

Daily challenge / streaks / achievements — Motivation can help, but streaks should not shame or create pressure. Offer rest days, streak freeze transparency, non-streak goals, and learner-controlled reminders. Ensure daily challenge date logic is timezone-safe

Printable sheets / study exports — Useful for offline study and teachers. Validate export layout for Nepali font embedding, page breaks, long text, dark-mode-independent print styles, and screen-reader-friendly source content

Progress analytics — Heatmaps and badges are engaging. Avoid treating time-on-app or streak length as learning mastery. Prioritize skill-level evidence (retrieval accuracy, delayed recall, speaking attempts, writing revisions) and clearly explain what each metric means

Evidence — src/components/DictionaryView.tsx; LecturePlayer.tsx; PracticeArenaView.tsx; MinimalPairLab.tsx; ArticulationGuideLab.tsx; DictationPassageStudio.tsx; AudioShadowingTrainer.tsx; WritingInterferenceLinter.tsx; DailyChallengeModal.tsx; ProgressSettingsView.tsx; PrintableStudySheetModal.tsx; src/services/storage.ts

Phase 5 — Security, privacy & Firebase

High priority: AI endpoints — server.ts exposes POST routes for lesson generation, sentence coach, word explainer and roleplay. Source review did not identify a server-side Firebase ID-token verification middleware or rate limiter protecting these endpoints. If publicly deployed as shown, unauthenticated callers may be able to consume the server's Gemini quota. Confirm actual hosting/network controls; do not assume frontend login protects an API

Required controls — Verify Firebase ID token on server; authorize user and plan; rate-limit by UID and IP; per-user daily quota; maximum input/history lengths; request size and concurrency caps; upstream timeout; safe error mapping; structured logs without sensitive prompts; abuse alerting; secrets only in server environment

Firebase rules — firestore.rules appears to enforce ownership and restrictive document shape. This is good defense-in-depth. However, firestore.rules.test.ts reads like test scenario descriptions rather than a demonstrated executable Emulator suite; security assertions must run, not merely be documented

Rules validation — Use Firebase Emulator + @firebase/rules-unit-testing. Test unauthenticated denial, cross-user read/write denial, verified/unverified user behavior, unexpected keys, type/size bounds, immutable fields, delete/reset and boundary values. Add CI job and rule coverage

Client configuration — firebase-applet-config.json is client configuration, not a secret by itself; Firebase web config is normally public. Security depends on Auth, Firestore rules, App Check where appropriate, quotas and API restrictions. Review exposed project identifiers and Google API key restrictions

Privacy — Publish a plain-language privacy notice: collected data, account identifiers, progress fields, AI request contents, retention, deletion/export, third-party processors and whether prompts are used by providers. Add account/data deletion flow and local-data clear/export controls. Do not log learner text or audio by default

Evidence — server.ts; src/services/firebase.ts; firestore.rules; firestore.rules.test.ts; firebase-applet-config.json; security_spec.md

Phase 6 — AI architecture, quality & cost

Strengths — Gemini requests use response schemas in at least roleplay flow; server-side key avoids shipping the Gemini secret to browser code. The AI features align with the product's bilingual learning context

Risks — AI route payloads should be validated beyond a single required text field. History can be user-controlled; prompts interpolate learner content; upstream output may be empty, malformed or semantically wrong. Raw error messages can leak implementation details. No quotas/rate limit visible in reviewed server code

Prompt & output controls — Use explicit system/user message separation; treat all learner text as untrusted; constrain output schema; validate JSON and field lengths; sanitize rendered content; reject invalid CEFR values; cap conversation history and tokens; make prompt versioned and testable

Cost governance — Define model per task; enforce maximum output tokens; set per-account/day budgets; cache deterministic explainers where safe; expose usage telemetry; alert on spikes; provide graceful fallback when AI quota is exhausted

Learning-quality evaluation — Build a golden test set of 100–200 learner inputs with expected correction principles. Score correctness, clarity, Nepali translation quality, level fit, tone, hallucination and harmful advice. Include Nepali-English code-switching and edge cases. Human linguist review remains necessary

UX — Explain AI limitations; distinguish generated feedback from verified curriculum; allow learner to report a bad correction; keep the original sentence visible; show concise correction plus explanation and optional deeper detail

Phase 7 — Offline, persistence & synchronization

Confirmed sync gap — POST /api/sync accepts events, counts them and returns a 'synced' response with a timestamp, but the inspected handler does not persist or process those events. The client flushPendingSyncQueue clears the queue after a successful HTTP response. This creates a false-success path and potential event loss

Fix — Either remove this event queue path if Firebase snapshot sync is the intended source of truth, or implement durable idempotent event storage. Each event needs ID, schema/version, user ID, createdAt, retry count and acknowledgement. Clear only acknowledged events; use exponential backoff and dead-letter/error visibility

Local storage — LocalStorage is the immediate state source and IndexedDB is also written. IndexedDB failures can resolve quietly; LocalStorage quota/write failures are caught but user-facing persistence state is not apparent. Define one canonical persistence strategy and explicit fallback

Merge correctness — Test offline edits followed by sign-in, simultaneous devices, stale cloud snapshots, duplicate events, account switch, sign-out, reset and partial failure. Use per-field/per-record timestamps or operation-based merge where overwrite/union semantics are insufficient. Protect destructive reset with confirmation and clear local+cloud consistently

PWA — Validate service-worker cache strategy for HTML/app shell, versioned assets and stale data. Ensure app updates are surfaced safely; never cache authenticated API responses or sensitive user data by default. Test first install, update, offline cold start and storage eviction

Evidence — src/services/storage.ts; src/services/firebase.ts; server.ts; public/sw.js; src/hooks/usePWA.ts

Phase 8 — Performance, responsive design & browser compatibility

Potential bottleneck — massiveLexicon.ts is a large corpus/generator and some practice components are very large. If all data and tools are eagerly imported, initial JavaScript and memory cost may rise. Actual bundle size and runtime performance have not been measured

Optimisation plan — Run production build and visualise bundle; lazy-load route/features; split static content; virtualize dictionary rows; debounce search; precompute normalized search fields; avoid repeated full-array transforms; memoize only measured hot paths; inspect React Profiler

Browser capability handling — SpeechSynthesis and SpeechRecognition support/voices differ by browser and OS. Detect support and permissions; provide clear fallback when unavailable; handle microphone denial, no speech result, network-based recognition and language mismatch

Performance budgets — Set budgets for initial JS, route chunks, LCP, INP, CLS and memory. Establish baseline on low-end Android and mid-range laptop, throttled network, and Safari/iOS. Avoid claiming a performance score until measured

Responsive QA — Check 320/375/768/1024/1440px widths, long Nepali strings, IPA wrapping, tables, dialogs, keyboard, zoom and orientation. Test touch target sizes and safe-area insets

Phase 9 — Testing, CI/CD & release discipline

Current scripts — package.json has dev/start/build/preview/clean/lint (lint runs tsc --noEmit). No dedicated test command is shown. No GitHub Actions workflow was present in the inspected tree

Test pyramid — Unit: SRS transitions, data validation, lexicon rules, linter patterns, progress merge. Component: forms, navigation, lesson player, dictionary filters, modal accessibility. Integration: Firebase emulator, AI route schemas, offline queue. E2E: onboarding→lesson→practice→progress; offline→online; login→sync; reset/export

CI pipeline — On PR: install locked dependencies, type-check, lint, unit tests, rules tests, build, dependency/security scan. On main: deploy preview/staging; smoke tests; release approval; production deploy; rollback path

Release checklist — Environment secrets configured; Firebase rules deployed and tested; AI quota configured; privacy/security review; PWA version; migration/backward compatibility; error monitoring; smoke tests; rollback plan; release notes

Phase 10 — Content integrity & pedagogy

Content is a product asset — Curriculum and lexicon quality may create greater learner value than adding more UI. IPA, phoneme labels, silent-letter segmentation, Nepali translation, grammar explanations and examples should be reviewed as structured content

Content QA pipeline — Define schemas and automated checks: duplicate IDs; missing translation; invalid CEFR; malformed IPA; impossible silent-letter segmentation; incorrect word/phoneme pairing; answer key mismatch; sentence length; missing audio metadata; unescaped HTML; source/author/review date

Pedagogical quality — Set measurable objective for each lesson; alternate explanation and retrieval practice; include spaced delayed recall; distinguish recognition from production; include transfer to real speech/writing; provide error-specific remediation

Cultural/language sensitivity — Use natural Nepali rather than literal machine translation; account for regional variation and honorific/register differences; explain accent intelligibility vs accent reduction; test examples with native Nepali speakers and qualified English phonetics educators

Editorial governance — Add content versioning, reviewer attribution, change log, learner feedback and a correction process. AI-generated lessons should be marked, reviewed or quality-scored before entering core curriculum

Phase 11 — Existing feature enhancements

- Dashboard: learner goal, continue-learning CTA, today's 5–10 minute plan, and clear progress summary; reduce competing cards
- Curriculum: lesson objectives, prerequisites, estimated time, mastery check, review schedule and resume marker
- Dictionary: learner-friendly filters, pronunciation audio alternatives, word family, collocations, example context, personal notes, offline favorites and import/export
- SRS: daily due queue, Again/Hard/Good/Easy, explainable intervals, error trends, and a recovery mode after missed days
- Pronunciation: calibrated reference recordings, syllable stress/intonation, minimal-pair listening before speaking, and safe confidence feedback
- Writing linter: confidence level, false-positive feedback, context-aware examples, grammar categories and personal error trend
- Roleplay: scenario difficulty, role/setting controls, turn replay, transcript, feedback toggle and end-of-session report
- Progress: skill-level chart, delayed recall, weekly review, learning streak alternatives and privacy-friendly export
- Settings: data export/delete, reminder consent, AI usage, audio/mic permissions, language preferences, accessibility and theme

Phase 12 — New feature opportunities & future work

1. Adaptive diagnostic & placement — A 10–15 minute diagnostic across listening, vocabulary, grammar, reading and speaking to recommend a starting path. Use confidence intervals and allow manual level choice; do not overclaim a formal CEFR certification

2. Personalized learning path — Use learner errors, time availability, goals and preferred practice modes to create a weekly plan. Explain why each activity is recommended and let learners override it

3. Pronunciation feedback with visual phoneme coaching — Add phoneme-level guidance, stress/intonation and minimal-pair recognition. Treat automated speech scores as approximate; prioritize actionable cues rather than a single accent score. Consider privacy-first on-device processing where feasible

4. Real-world scenario library — Expand roleplay to Australian workplace, healthcare, childcare, customer service, university, rental, transport and everyday conversations; include formal/informal registers and Nepali explanations

5. Writing coach & progression portfolio — Save drafts, revisions and error patterns; show before/after improvement; support email, workplace note, academic paragraph and conversational writing

6. Listening comprehension with authentic accents — Curated audio across Australian, British, American and South Asian English speakers; speed control, transcript reveal, comprehension questions and accent-intelligibility education

7. Teacher/mentor mode — Classroom groups, assignment links, progress snapshots, printable lesson packs and educator dashboard with explicit consent and minimal learner data

8. Community practice / language exchange — Opt-in moderated pair practice, topic prompts, safety controls and reporting. Avoid exposing personal details by default; design for child-safety and abuse prevention if minors are permitted

9. Offline content packs — Download selected lessons/audio/translations and sync completed activities later. Show storage size, expiry/version and sync status clearly

10. Learner-owned data & portability — Export/import learning history, personal vocabulary, mistakes and settings in a documented JSON/CSV format; provide account deletion and clear retention controls

11. Evidence-based learning analytics — Measure delayed retention, transfer and skill-specific improvement—not only time, streaks or completed screens. Conduct controlled pilot studies with informed consent

12. Multilingual bridge expansion — Potential future support for other learner language backgrounds, but only after a content and linguistic QA pipeline exists. Nepali should remain the first-class design target rather than diluted by generic translations

Suggested product experiments: (a) guided onboarding vs current entry; (b) short daily plan vs open-ended dashboard; (c) feedback detail level; (d) reminder frequency with opt-in; (e) delayed recall and retention after 7/30 days. Track completion, learning gain, frustration and accessibility—not engagement alone

Roadmap & prioritisation

P0 — immediate (0–2 weeks) — Server-side auth for AI routes; rate limits/quotas; input caps; remove or fix fake sync acknowledgement; confirm deployed server exposure; review secret handling and Firebase rules

P1 — reliability (2–6 weeks) — Executable Firestore rules tests; persistence/merge test matrix; AI schema and error tests; data validation; CI typecheck/build/test; error monitoring; account export/delete design

P2 — quality (6–12 weeks) — App.tsx decomposition; accessibility remediation; bundle analysis/lazy loading; responsive QA; content QA scripts; usability tests; improve dashboard and SRS

P3 — product evolution (quarter+) — Adaptive placement; personalised path; richer pronunciation feedback; scenario expansion; offline packs; teacher mode; evidence-based analytics pilot

Recommended ownership: one engineering owner for API/security and data reliability; one frontend owner for modularisation/accessibility; a content/linguistics reviewer for corpus and AI evaluation; and a product/UX owner for learner testing. If team size is one, do P0/P1 sequentially before new feature work

Scorecard & conclusion

Preliminary scores (heuristic, source-only; not benchmark or test results):

Feature breadth — 88/100 — broad and coherent set of learning tools

Product differentiation — 82/100 — Nepali-specific phonology/grammar bridge is a strong niche

Architecture/maintainability — 68/100 — useful separation, but App.tsx and large components need decomposition

Security posture — 58/100 — Firestore rules direction is positive; API protection and validation need verification

Offline/data reliability — 55/100 — dual local storage strategy; sync acknowledgement issue and silent failure concerns

Performance — 67/100 — plausible optimization opportunities; no measurements

Accessibility — 45/100 — no demonstrated comprehensive audit; required manual checks

Testing/CI — 30/100 — scripts exist, automated test coverage/CI not demonstrated

Content quality assurance — 65/100 — strong curated concept; systematic QA/review evidence limited

Production readiness — 57/100 — many foundations, but key security/reliability/test gates remain

Overall provisional score: 62/100. This is not a rating of the founder's effort or the app's potential; it is a risk-weighted estimate of current engineering readiness from the reviewed source. A runtime, deployment, rules and user study could materially change it

Bottom line: BhashaSetu has a compelling niche and a strong feature base. The highest-return work now is to make learning progress trustworthy, AI usage safe and affordable, content dependable, and the experience accessible and measurable. Add new features after the reliability foundation is established.

Appendix — source map

- README.md — product overview, curriculum and design philosophy
- package.json — dependencies and available scripts
- src/App.tsx — app shell and major state orchestration
- src/data/syllabus.ts; curriculumDetails.ts; lectureEnhancements.ts — curriculum content
- src/data/massiveLexicon.ts — dictionary corpus/generator
- src/components/DictionaryView.tsx — dictionary and vocabulary workflows
- src/components/LecturePlayer.tsx — lesson interaction
- src/components/PracticeArenaView.tsx — practice tools
- src/components/MinimalPairLab.tsx; ArticulationGuideLab.tsx — phonetics practice
- src/components/DictationPassageStudio.tsx; AudioShadowingTrainer.tsx — listening/speaking practice
- src/components/WritingInterferenceLinter.tsx — writing feedback rules
- src/components/ProgressSettingsView.tsx; DailyChallengeModal.tsx — progress/motivation
- src/services/storage.ts — local state, SRS and queue
- src/services/firebase.ts — authentication and cloud progress
- src/services/speech.ts — browser speech functions
- server.ts — Express and Gemini endpoints
- firestore.rules; firestore.rules.test.ts; security_spec.md — database access control and security notes
- src/hooks/usePWA.ts; public/sw.js; public/manifest.json — install/offline behavior